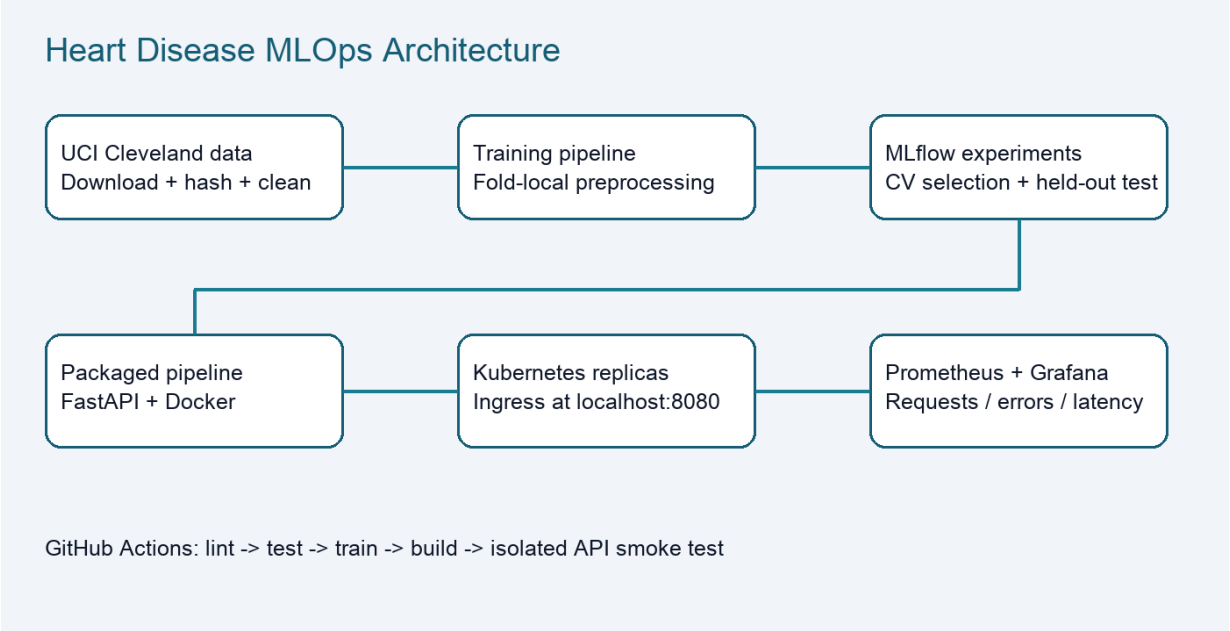
1. Project objective and architecture

Machine Learning Operations (AIMLCZG523), Assignment 01. End-to-end educational classification of heart disease presence using the UCI Cleveland dataset. Prepared 1 October 2026.

The system acquires a traceable dataset, creates reproducible experiments, packages preprocessing with the model, and exposes validated inference through FastAPI. Automation checks source quality, unit behavior and container serving. Kubernetes and monitoring configurations support deployment.



Evidence policy: this report labels configurations separately from executed checks. This dataset classifier predicts a historical disease label; it is not a validated clinical diagnostic service or future-risk score. Review and personalize this work in accordance with the course's academic integrity and AI-assistance policies.

2. Dataset acquisition and provenance

Source: UCI Heart Disease, https://archive.ics.uci.edu/dataset/45/heart+disease. Citation: Janosi, Steinbrunn, Pfisterer and Detrano (1989), DOI 10.24432/C52P4X. License: CC BY 4.0.

The processed Cleveland subset contains 303 rows and 13 predictors plus the original num target. The assignment's reference to 14+ features is interpreted as the commonly used 14-column dataset, including the target; no artificial predictors were added.

scripts/download_data.py downloads the original file only if absent, validates its structure, writes a cleaned CSV and records a SHA-256 digest. It removes exact duplicates and maps num=0 to absence and num=1..4 to presence.

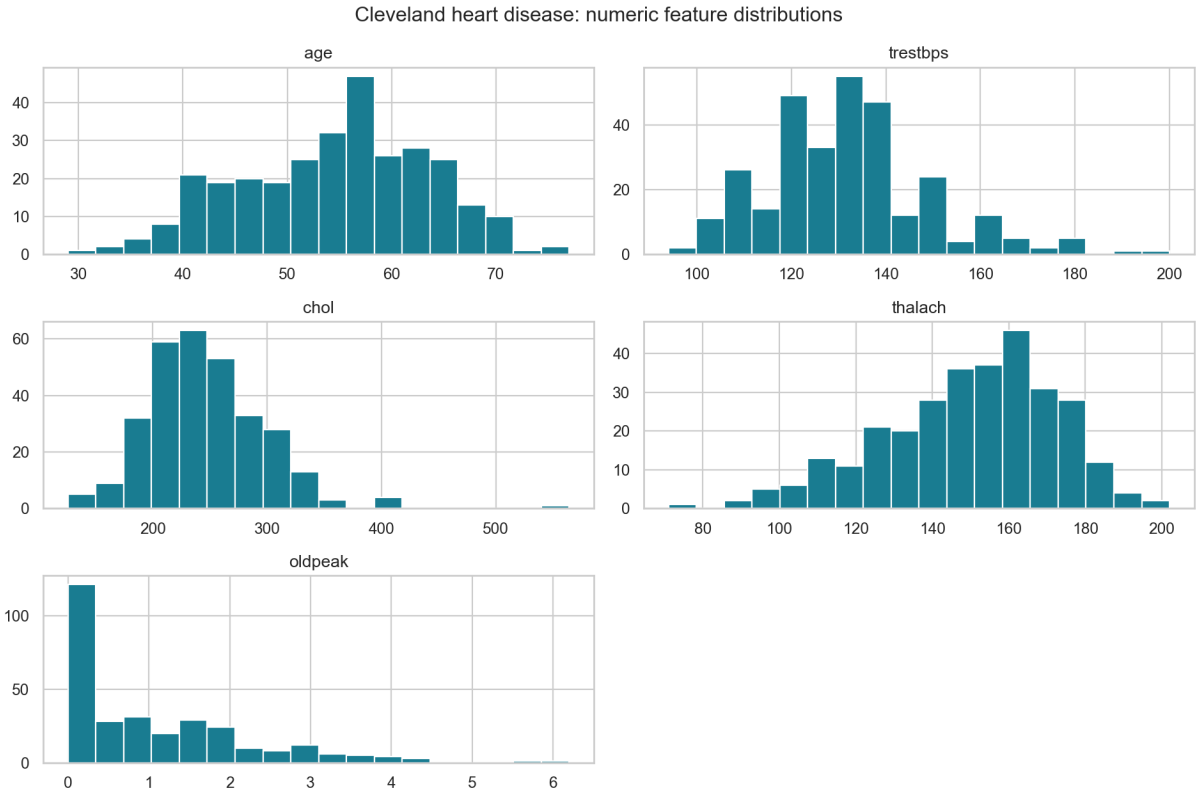
Data property	Observed value
Raw / clean rows	303 / 303
Duplicates removed	0
Missing ca / thal	4 / 2
Target mapping	0 -> absent; 1,2,3,4 -> present

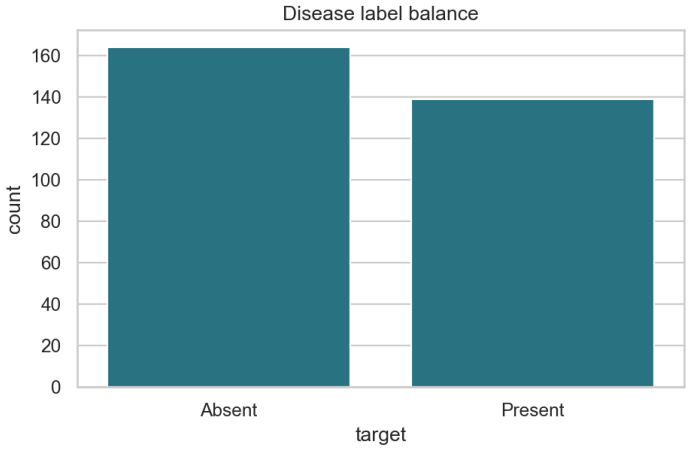
Raw data checksum: a74b7efa387bc9d108d7d0115d831fe9b414b29ae7124f331b622b4efa0427c8

Categorical values retain original UCI coding: cp 1-4, slope 1-3, ca 0-3 and thal 3/6/7. Predictors include age, sex, resting blood pressure, cholesterol, fasting blood sugar, ECG, maximum heart rate, exercise angina and ST depression. Full feature descriptions are linked in the README.

3. Exploratory analysis and cleaning

EDA is reproduced with scripts/eda.py or notebooks/01_eda.ipynb. Histograms examine distributions and potential outliers; class counts reveal label balance; the correlation heatmap describes associations without being used for test-informed feature selection.





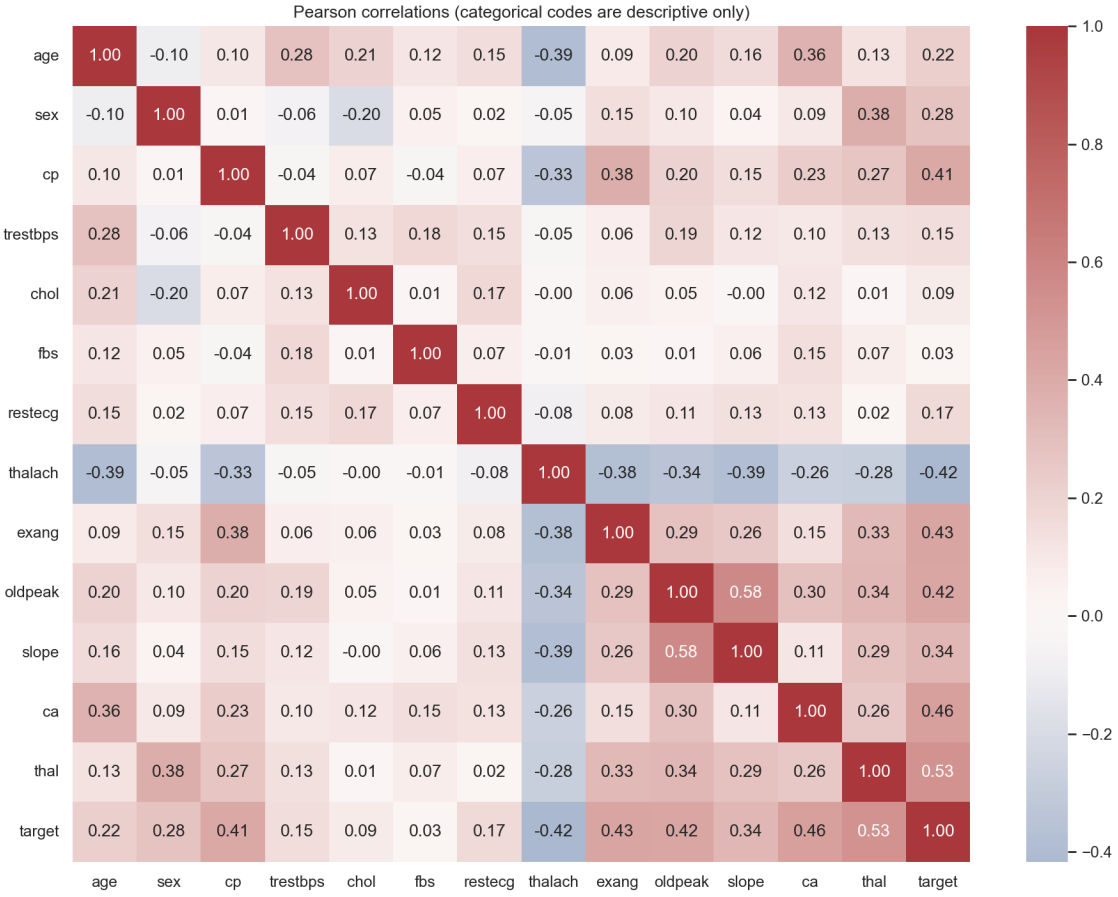
Missing predictors remain missing in the cleaned CSV. No global imputation, scaling or arbitrary outlier deletion is performed. This preserves the distinction between deterministic cleaning and learned preprocessing. Numeric category correlations are descriptive and should not be interpreted as continuous clinical effects.

4. Features and leakage prevention

The preprocessing ColumnTransformer handles five numerical variables (age, trestbps, chol, thalach, oldpeak) with median imputation and StandardScaler. Eight categorical predictors use mode imputation and OneHotEncoder(handle_unknown='ignore').

Both estimators receive the same feature representation for a controlled comparison. Scaling is useful for Logistic Regression; it is unnecessary but harmless for Random Forest. All 13 source predictors are retained; no selection procedure is fitted on the held-out test set.

Stratified splitting uses seed 42, 242 training rows and 61 test rows. The exact indices are saved in artifacts/split.json. Five-fold shuffled stratified CV operates exclusively on the training partition.



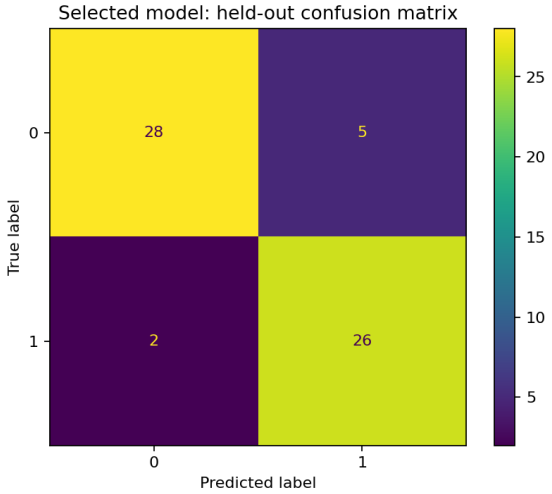
Imputation and encoding are fitted within each CV fold because they are inside the sklearn Pipeline. Tests check training-only numeric imputation statistics and serialized pipeline equivalence.

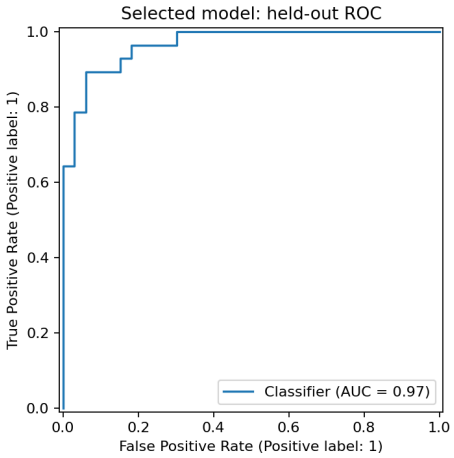
5. Model tuning and held-out evaluation

Logistic Regression tests C in {0.1,1,10}. Random Forest uses 150 trees, max_depth in {3,None} and min_samples_leaf in {1,4}. Seven candidate configurations are assessed using the same five folds. Selection maximizes mean CV ROC-AUC rather than examining test performance.

Estimator	CV AUC	CV accuracy	CV recall
logistic_regression	0.9025	0.8471	0.7834
random_forest	0.8992	0.8138	0.7462

Selected estimator: logistic_regression. Its held-out metrics are: accuracy=0.8852; precision=0.8387; recall=0.9286; roc_auc=0.9665.





The decision threshold is fixed at 0.5. Recall and precision expose different error costs; the report does not tune a threshold against the test set. A small 61-row test partition means these scores have substantial uncertainty. No external validation or probability calibration has been performed.

6. Experiment tracking and reproducibility

MLflow uses a local SQLite tracking store (mlflow.db) and local artifact storage. Each candidate logs hyperparameters, seed, dataset hash, five-fold metric means and standard deviations, CV result tables and EDA plots. The final run logs held-out metrics, confusion matrix, ROC plot and the full sklearn pipeline.

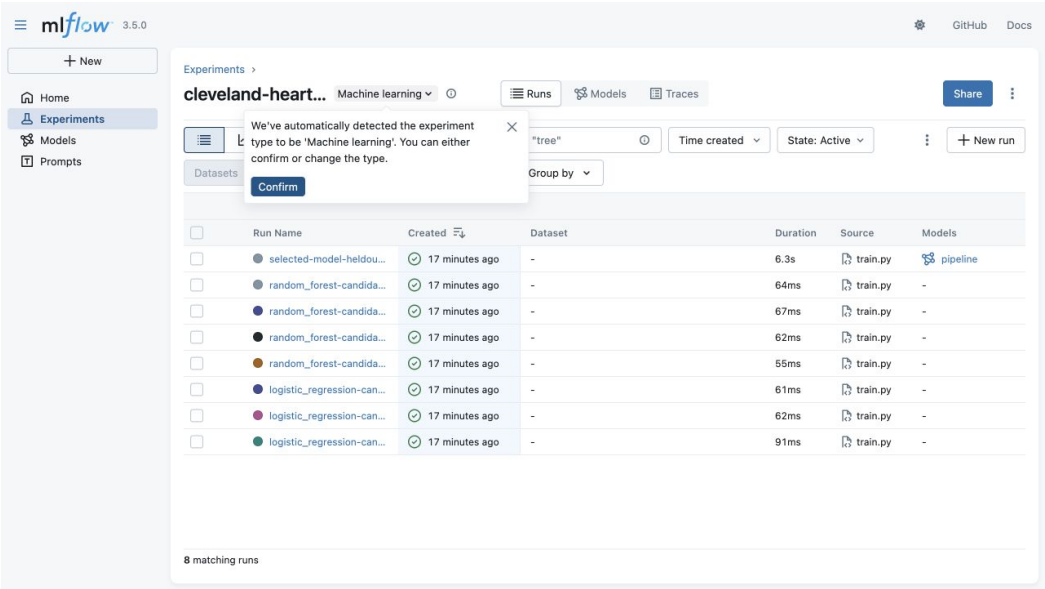
Final evaluation run ID: ce00050d09b24a168a9da3cb91c5410b

View experiments using: mlflow ui --backend-store-uri sqlite:///mlflow.db --host 127.0.0.1 --port 5000. The database and artifacts must remain together when transferred. CI uploads both as downloadable workflow artifacts.

artifacts/model.joblib contains preprocessing and the selected estimator. Serving reuses this fitted pipeline and never duplicates preprocessing logic. The evaluated model is retained without refitting on held-out data.

requirements.txt pins direct packages; requirements-lock.txt records the exact local resolved environment for audit. The project is installed as an editable Python package through pyproject.toml. Linux CI and Docker perform independent installs from the direct manifest.

For reproducibility, retain the raw dataset, data hash, split indices, CV tables, MLflow run and package manifests. Random seeds improve repeatability but do not guarantee identical results across all platforms and dependency implementations.



7. API serving and container packaging

FastAPI exposes POST /predict, GET /health, GET /metrics and interactive /docs. Pydantic rejects missing fields, invalid category codes, unknown fields and non-finite numeric values. ca and thal accept explicit null values that the fitted pipeline imputes.

The prediction response contains the binary label, disease_probability, confidence, model version and educational-use message. Confidence is max(p,1-p), the probability assigned to the predicted label; it is not calibrated clinical certainty.

The model is loaded once during application startup. Missing model artifacts fail startup visibly. Request logging records method, route, status and latency without recording the patient payload.

Docker uses python:3.13-slim, installs the project, copies the model and metadata, serves on port 8000 and runs as a non-root user. A Docker healthcheck calls /health. The smoke-test script builds and starts the image, checks health and prediction and saves logs.

Example: curl http://localhost:8000/predict -H 'Content-Type: application/json' --data @examples/patient.json

Container execution status: Image built; isolated health and prediction passed

Actual local API response and metrics are saved under artifacts/evidence. Never substitute host Python checks for proof that the container builds and runs.



Deployment evidence: recorded live kubectl output shows two ready API replicas and the configured Ingress. The full HTTP response is saved in artifacts/evidence/kubernetes-prediction.json.

8. Testing and continuous integration

Pytest covers label conversion, missing-value preservation, duplicate removal, invalid dataset categories, target validation, fold-safe preprocessing, serialization equivalence, valid inference, input errors, nullable predictors, health and metrics.

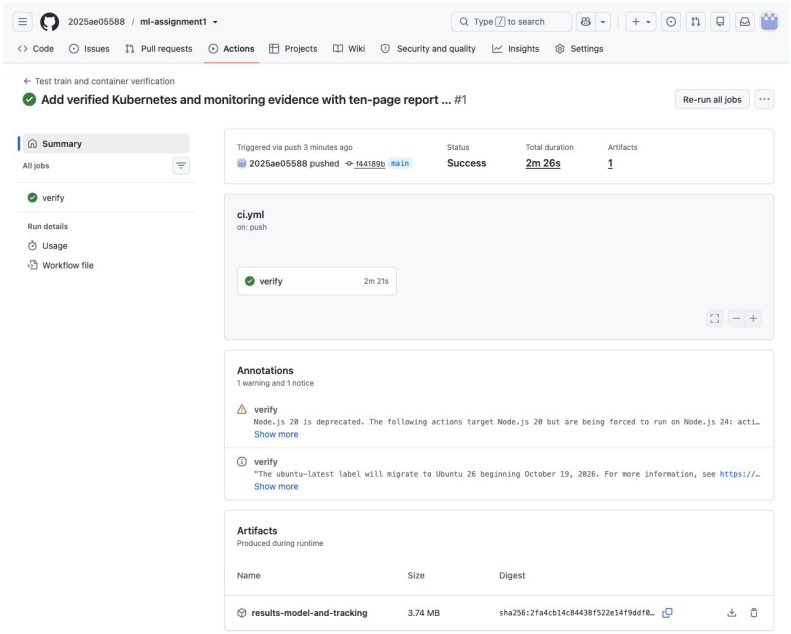
Local test outcome: 11 passed

Ruff performs syntax/import/undefined-name checks. GitHub Actions installs Python 3.13 and dependencies, prepares data, runs lint and tests, generates EDA, trains and tracks experiments, then builds and smoke-tests the Docker image.

Command errors stop the workflow. An always-running upload step preserves metrics, model files, test XML, MLflow storage and container logs. Unit tests are a gate before training; training itself raises errors for failed CV fits.

Repository published: https://github.com/2025ae05588/ml-assignment1. GitHub Actions run 36829271226 succeeded, including all quality, training and isolated container checks. Model/results/tracking artifacts were uploaded. Actual status, logs and screenshots are included.

The CI pipeline verifies the package; local Kubernetes deployment uses the included script. Cluster credentials stay excluded from Git.



9. Kubernetes deployment and monitoring

deployment/kubernetes.yaml creates a namespace, two API replicas and a LoadBalancer Service on port 80. Readiness/liveness probes use /health. Resource requests/limits, non-root execution, dropped capabilities and a read-only filesystem provide a basic deployment configuration.

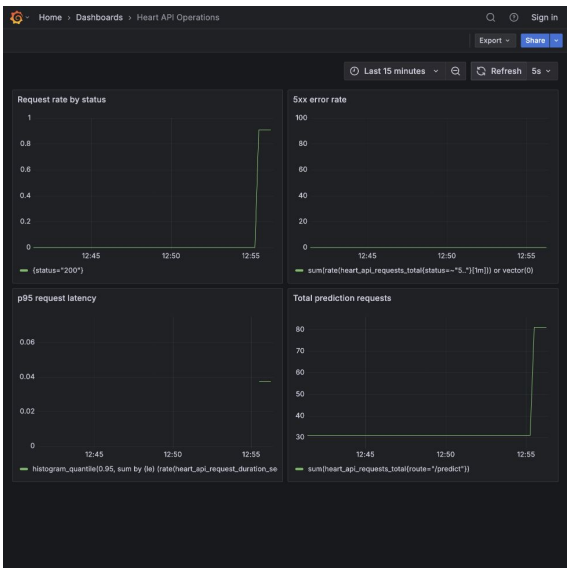
Verified local route: kind with a mapped NodePort and Traefik Ingress at http://127.0.0.1:8080. Reproduce with bash scripts/deploy_kind.sh. For Minikube, run scripts/deploy_local.sh and minikube tunnel. Inspect the Service external address and call /health and /predict through that address. A cloud cluster additionally requires an accessible registry image and credentials.

Kubernetes execution status: kind cluster; two API replicas ready; Traefik Ingress verified at http://127.0.0.1:8080

Docker Compose provisions FastAPI, Prometheus and Grafana. Prometheus scrapes /metrics every five seconds. The Grafana dashboard shows request rate, 5xx error rate, p95 latency and total prediction count. Generate traffic to demonstrate changes.

Monitoring execution status: Compose services running; Prometheus target up and request series verified; Grafana healthy. The stack runs on localhost ports 8002 (API), 9090 (Prometheus) and 3000 (Grafana). The Compose monitoring demonstration targets its API container, not an unconfigured Kubernetes endpoint.

Operational limitations: no authentication, TLS termination, drift detection, retraining trigger or production alerting is implemented. These are future extensions. Monitoring measures service behavior, not clinical accuracy. Do not expose the demonstration stack publicly without appropriate access controls.



10. Reproduction and submission checklist

Setup: python3 -m venv .venv; activate it; pip install -e .; python scripts/download_data.py; python scripts/eda.py; python scripts/train.py; pytest -q; ruff check .; uvicorn heart_ml.api:app --port 8000.

Repository deliverables: source, download and training scripts, EDA notebook, raw and cleaned data with citation, requirements files, tests, workflow YAML, Dockerfile, Kubernetes manifests, monitoring configuration, screenshots and this ten-page report.

Submission evidence: actual MLflow experiment view; successful CI logs/artifacts; Docker build and sample prediction; Kubernetes replicas and exposed endpoint; Grafana traffic metrics; short live video of the whole pipeline.

GitHub repository: https://github.com/2025ae05588/ml-assignment1. Verified deployed API access: http://127.0.0.1:8080 through the local kind cluster Ingress. No public cloud URL is claimed.

reports/DEMO_SCRIPT.md supplies the live recording sequence. reports/pipeline_terminal_demo.mp4 records actual deployment/monitoring command outputs and genuine browser screenshots. Raw timestamps and exit codes are in demo_commands.json. It is a terminal-output recording, not an operating-system desktop capture.

All implementation stages have executed successfully: local data/model/API verification, remote CI, Docker isolation, Kubernetes Ingress and monitoring. Genuine screenshots and the terminal demonstration are included. Record an oral desktop presentation separately if your instructor requires that format. reports/COMPLETION_STATUS.md and artifacts/evidence/verification.json distinguish prepared and executed work.

References: UCI Heart Disease (DOI 10.24432/C52P4X); sklearn Pipeline and GridSearchCV; MLflow tracking; FastAPI; Docker; Kubernetes; Prometheus and Grafana official documentation. Refer to README commands and implementation files for concrete reproduction details.